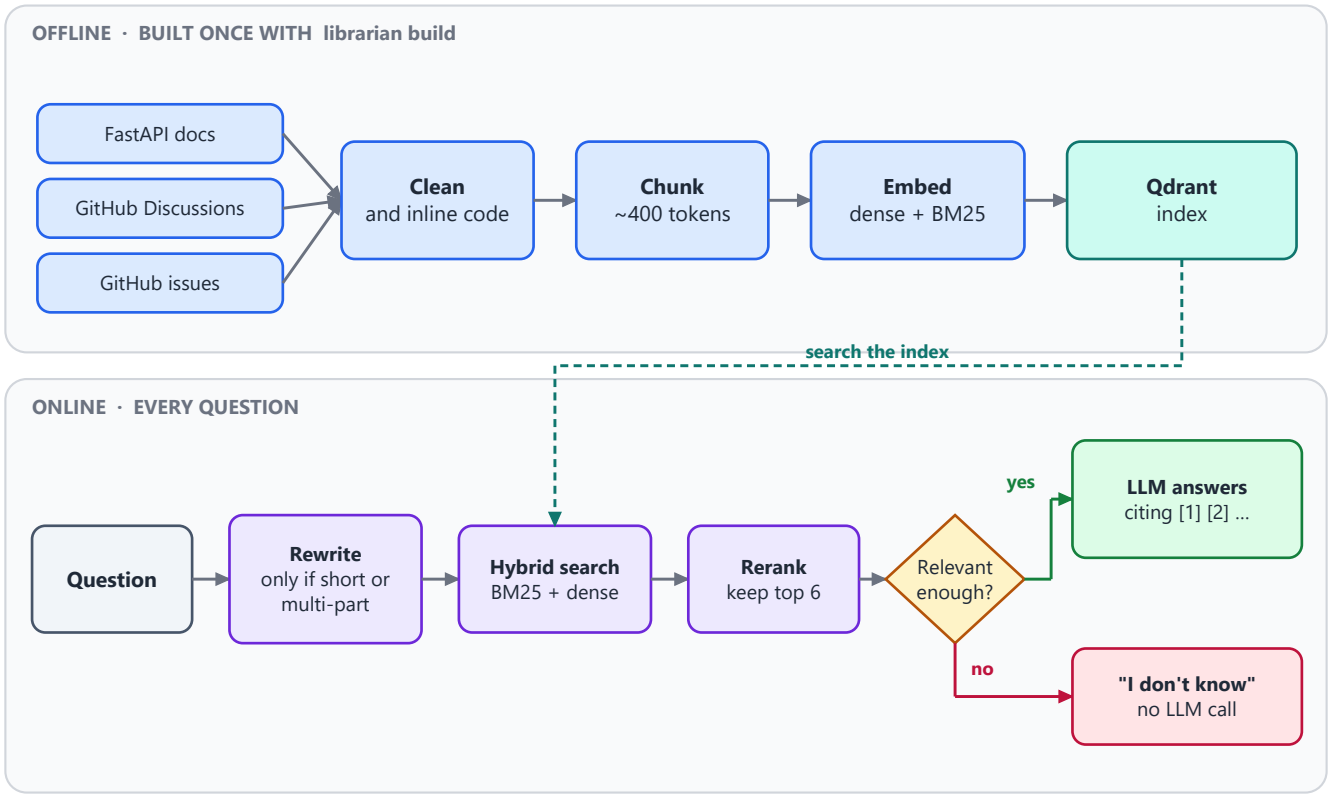
The Librarian

A RAG service that answers FastAPI questions from the official docs, GitHub Discussions and issues, citing a source for every claim or saying "I don't know".

Workflow



Offline (once): collect, clean, chunk, embed and index the corpus. Online (every question): rewrite if needed, hybrid search, rerank, then answer with citations or abstain.

Files and technologies

AREA	FILE	TECHNOLOGY USED	ROLE
Setup	`pyproject.toml`	Python packaging (setuptools), ruff, pytest	Dependencies + the librarian command
	`.env.example / config.py`	pydantic-settings, environment variables	All settings in one place
	`models.py`	Python dataclasses, JSON Lines	Document and Chunk data types
Data	`ingest/docs_loader.py`	httpx, tarball download, regex markdown cleaning	Load FastAPI docs, inline code examples
	`ingest/github_loader.py`	GitHub GraphQL + REST API, disk cache	Load Discussions and issues (no usernames)
	`chunking.py`	Structure-aware markdown chunking, tiktoken (400 tokens, 60 overlap)	Split documents into chunks
	`embeddings.py`	fastembed / ONNX: bge-small-en-v1.5, BM25, MiniLM cross-encoder	Dense + sparse vectors, reranker
	`store.py`	Qdrant vector database (dense + sparse + metadata filters)	Index and search chunks

AREA	FILE	TECHNOLOGY USED	ROLE
Search	`rewrite.py`	LLM query rewriting (ALLaM-2-7B, JSON mode)	Fix short / multi-part questions
	`retrieval.py`	Hybrid BM25 + dense, Reciprocal Rank Fusion, cross-encoder reranking	Find the 6 best chunks
Answer	`llm.py`	OpenAI-compatible client (Groq), token + cost tracking	Talk to any LLM provider
	`generate.py`	Grounded prompting, citation parsing + validation (gpt-oss-120b)	Answer with [n] citations
	`pipeline.py`	RAG orchestration, abstention threshold	Question -> answer or "I don't know"
	`telemetry.py`	SQLite	Log requests, feedback, latency, cost
Interface	`cli.py`	Typer	build / ingest / index / ask / serve
	`api/app.py, main.py`	FastAPI, Uvicorn, CORS, rate limiting	HTTP API
	`api/schemas.py`	Pydantic	Validate requests and responses
	`api/static/*`	HTML, CSS, JavaScript, marked, DOMPurify	Web page with feedback buttons
Evaluation	`data/eval/questions.jsonl`	Hand-written gold Q&A set	25 test questions (5 unanswerable)
	`eval/retrieval_eval.py, ablation.py`	recall@k, MRR, ablation study	Measure search quality
	`eval/generation_eval.py`	LLM-as-judge (Qwen3.8-27B)	Correctness, faithfulness, citations
	`eval/build_dataset.py, load_test.py`	LLM data generation, concurrent HTTP load test	More questions; p50/p95 latency
	`tests/*`	pytest	Unit tests
Deploy	`Dockerfile, docker-compose.yml`	Docker, Docker Compose	Run app + Qdrant anywhere
	`.github/workflows/ci.yml`	GitHub Actions	Lint, test, build on every push